



JavaScript

Axios vs Fetch API



API Handling



Swipe

Shivendra Dwivedi
[@shivendra-dwivedi](https://twitter.com/shivendra-dwivedi)



1: API Creation / Call Syntax

Axios: There are 3 ways to do this in axios

1: Main axios() method: axios(config)

```
export const handleGetData = async () => {  
  try {  
    const response = await axios({  
      method: "GET",  
      url: "/api/v1/movies",  
      baseURL: "http://localhost:3000",  
    })  
    console.log(response);  
  } catch (err) {  
    console.error(err.message);  
  }  
};
```

2: Axios shorthand methods: axios.get(url [, config]),
axios.post(url [, data [, config]]), etc.

```
export const handleGetData = async () => {  
  try {  
    const response = await axios.get("/api/v1/movies", {  
      baseURL: "http://localhost:3000",  
    });  
    console.log(response);  
  } catch (err) {  
    console.error(err.message);  
  }  
};
```



3: Axios custom instance methods: `axios.create(config)`

```
const api = axios.create({ baseURL: "http://localhost:3000" });
export const handleGetData = async () => {
  try {
    const response = await api.get("/api/v1/movies"); ← Step 1
    console.log(response); ← Step 2
  } catch (err) {
    console.error(err.message);
  }
};
```

fetch API: It has only 1 way to do API call.

`fetch(url, [options])`

```
const BASE_URL = "http://localhost:3000";
export const handleGetData = async () => {
  try {
    let response = await fetch(`${BASE_URL}/api/v1/movies`); ← Step 1
    if (!response.ok) throw new Error(response.statusText);

    let data = await response.json(); ← Step 2
    console.log(data); ← Step 3
  } catch (err) {
    console.error(err.message);
  }
};
```

Axios = **Cleaner** + **Reusable** + **Scalable**

Fetch = **Low-level**, needs more boilerplate code.



2: Default Setting:

Axios: Here we can set default options at two levels **global** and **instance level**. Default options will be applied on all the API calls that we will make.

Global Level:

```
axios.defaults.baseURL = "http://localhost:3000";
axios.defaults.headers.common["Authorization"] = localStorage.getItem("token");

export const handleGetData = async () => {
  try {
    const response = await axios.get("/api/v1/movies");
    console.log(response);
  } catch (err) {
    console.error(err.message);
  }
};
```

Instance Level:

```
const api = axios.create({ baseURL: "http://localhost:3000",
  headers: {
    "Authorization": localStorage.getItem("token"),
  }
});

export const handleGetData = async () => {
  try {
    const response = await api.get("/api/v1/movies");
    console.log(response);
  } catch (err) {
    console.error(err.message);
  }
};
```




fetch API:

No default setting option is available. We must **repeat** headers & config in every request.

```
const BASE_URL = "http://localhost:3000";
const headers = {
  'Authorization': token,
  'Content-Type': 'application/json'
}
export const handleGetData = async () => {
  try {
    let response = await fetch(`${BASE_URL}/api/v1/movies`, {
      method: "GET",
      headers ← Include headers in each API call manually
    });
    if (!response.ok) throw new Error(response.statusText);

    let data = await response.json();
    console.log(data);
  } catch (err) {
    console.error(err.message);
  }
};
```

Axios = **DRY** (Don't Repeat Yourself)
Fetch = **Manual setup** in every request



3: Interceptors:

Axios: Interceptors are functions that run before a request is sent or before a response is handled by then() or catch().

Think of them as middleware for HTTP requests.

```
const api = axios.create({ baseURL: "http://localhost:3000" });
const tokenInterceptor = api.interceptors.request.use( ← Request Interceptor
  (config) => {
    const token = localStorage.getItem("token");

    const isPublic = config.method === "get" && new URL(config.url, api.defaults.baseURL).pathname === "/api/v1/movies";

    if (!isPublic && token) {
      config.headers.Authorization = token;
    }

    return config;
  },
  (err) => {
    alert("Some error occurred", err);
  }
);

const errorHandlerInterceptor = api.interceptors.response.use( ← Response Interceptor
  (response) => response,
  (err) => {
    if (err.response && err.response.status === 403) {
      console.log("Unauthorised, token doesn't match");
    }

    return Promise.reject(err);
  }
);
```

fetch API: No built-in interceptor present in fetch API.



4: Configuration Options:

Axios:

Configuration options let us customize how our API call behaves, like setting **timeouts**, **transforming data**, **showing progress**, etc.

1: transformRequest / transformResponse:

transformRequest used to **alter the data** which will be **send to the server** before it's being send.

```
const ai = axios.create({
  baseURL: "http://localhost:3000",

  transformRequest: [
    function (data, headers) {
      if (data instanceof FormData) {
        delete headers["Content-Type"]
        data.append("timestamp", "18/06/25");
        return data;
      }

      if (data && typeof data === "object") {
        data.timestamp = "17/6/2025";
        return JSON.stringify(data);
      }

      return data;
    },
  ],
});
```

adding a timestamp(hard coded) in the data before it is send to the server



```
transformResponse: [  
  function (data, headers) {  
    try {  
      return JSON.parse(data);  
    } catch (e) {  
      return data;  
    }  
  },  
],
```

← parsing the data before
it's handled by the then()
or catch() blocks

2: onUploadProgress / onDownloadProgress:

With these we can **track upload and download progress** in **axios**.

```
onUploadProgress: ({ total, progress, loaded }) => {  
  console.log(  
    "Upload ",  
    "progress:" + " " + progress,  
    "Total: " + total,  
    loaded  
  );  
},
```

← tracking upload progress

← Total number of bytes to
be uploaded

← How many bytes have been uploaded so far.

```
onDownloadProgress: ({ total, progress, loaded }) => {  
  console.log(  
    "download ",  
    "progress:" + " " + progress,  
    "Total: " + total,  
    loaded  
  );  
},
```

← tracking download
progress



3: timeout: Defines the **maximum time (in ms)** to wait for a response before canceling the request. If the server doesn't respond **within that time**, the request is canceled internally.

```
timeout: 3000,
```

fetch API: Fetch supports basic options: method, headers, body, signal (for manual cancellation).

It **doesn't have in-built** advanced options like **timeout**, **onUploadProgress**, **onDownloadProgress**, **baseUrl**, **transformRequest/Response**, etc.

```
export const handleWaitApi2 = async () => {
  const controller = new AbortController();
  const timer = setTimeout(controller.abort, 4000);

  try {
    let response = await fetch(`${BASE_URL}/api/waitBaby`, {
      method: "GET",
      signal: controller.signal,
    });

    if (!response.ok) throw new Error(response.statusText);
    clearTimeout(timer);
    const data = await response.json();
    alert(data.message);
  } catch (err) {}
};
```

here trying to mimic
timeout feature in fetch
API



5: Error handling:

Axios: Simple & Developer-Friendly. Axios **automatically throws errors** for: Network issues, Timeout
Non-2xx HTTP status codes (including 404, 500).

```
export const handleGetByIdData = async (id) => {  
  try {  
    let response = await ai.get(`/api/v1/movies/${id}`);  
    console.log(response);  
  } catch (err) {  
    console.error(err.message);  
  }  
};
```

fetch API: Fetch **does not throw errors** for HTTP failures like 404 or 500. Only throws on network failures (e.g., no internet). We must **check response.ok** manually.

```
export const handleGetByIdData = async (id) => {  
  try {  
    let response = await fetch(`${BASE_URL}/api/v1/movies/${id}`);  
    if (!response.ok) throw new Error(response.statusText);  
  
    const data = await response.json();  
    console.log(data);  
  } catch (err) {  
    console.error(err.message);  
  }  
};
```

manually throwing error.
'ok' property is true only
for 2xx range responses.



6: JSON Handling:

Axios: It handles JSON for you in both directions (Request / Response) :

Auto stringifies request body, **Auto parses** JSON response.
No need to manually call `JSON.stringify()` or `res.json()`.

```
let response = await axios.post('/api', { name: 'Shivendra' })
console.log(response);
```

fetch API: Must handle **all manually**:

- **Manually stringify** the body
- **Manually parse** response with `res.json()`

```
fetch('/api', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ name: 'Shivendra' })
})
.then(res => res.json())
.then(data => console.log(data))
.catch(err => console.error(err));
```

← manually handling



7: API call Canceling:

Axios: Axios previously used `CancelToken`, but it's now **deprecated**. Modern Axios from v0.22.0 uses the standard **`AbortController`**, same as Fetch:

```
const controller = new AbortController();

export const handleWaitApi = async () => {
  try {
    let response = await ai.get("/api/waitBaby", {
      signal: controller.signal,
    });
    alert(response.data.message);
  } catch (err) {}
};

export const abort = () => {
  controller.abort();
}
```

fetch API: Always use **`AbortController`** to cancel calls.

```
const controller = new AbortController();

export const handleWaitApi = async () => {
  try {
    let response = await fetch(`${BASE_URL}/api/waitBaby`, {
      method: "GET",
      signal: controller.signal,
    });
    const data = await response.json();
    alert(data.message);
  } catch (err) {}
};

export const abort = () => {
  controller.abort();
};
```




8: Backend compatibility:

Axios:

Axios was **not always available natively** in the backend (Node.js). Axios has always been a third-party HTTP client library that needs to be installed manually.

```
ktop\Axios Docs\practice-2 Axios> npm install axios
```

```
const express = require('express');
const axios = require('axios');
const app = express();

app.get('/get-data', async (req, res) => {
  try {
    const response = await axios.get('https://api.example.com/data');
    res.json(response.data);
  } catch (error) {
    res.status(500).json({ error: "Axios request failed!" });
  }
});

app.listen(3000, () => console.log('Server running'));
```

axios GET API call

fetch API: Since Node.js 18+, the Fetch API is built-in, so we can use it directly. But, if you're **using Node.js < 18**, you need to install a Fetch-compatible library like **node-fetch**.



```
import express from 'express';
const app = express();

app.get('/fetch-example', async (req, res) => {
  try {
    const response = await fetch('https://api.example.com/data');
    const data = await response.json();
    res.json(data);
  } catch (error) {
    res.status(500).json({ error: 'Failed to fetch data' });
  }
});

app.listen(3000, () => console.log('Server running on port 3000'));
```

Node.js v = 18+

```
\\Desktop\\Axios Docs\\practice-2 Axios> npm install node-fetch
```

```
const express = require('express');
const fetch = require('node-fetch'); // Import node-fetch
const app = express();

app.get('/fetch-example', async (req, res) => {
  try {
    const response = await fetch('https://api.example.com/data');
    const data = await response.json();
    res.json(data);
  } catch (error) {
    res.status(500).json({ error: 'Fetch failed' });
  }
});

app.listen(3000, () => console.log('Server running'));
```

Node.js v < 18



JavaScript

Was this helpful?



Like, **Comment** & **Follow**
for more JavaScript insights!



Shivendra Dwivedi
[@Shivendra-Dwivedi](#)



Connect

↕ Repost